

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
МАТЕМАТИЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Звіт з лабораторної роботи № 10
з «Конструювання програмного забезпечення»

Виконав: студент групи
6.1214-пі2

Кривокоритов Олександр
Перевірив: Мильцев Олександр
Михайлович

Запоріжжя 2026

Код

https://github.com/AlexKrivokorytov/lab10_cons
[tr](#)

1.4.2 Короткі теоретичні відомості

Поведінкові шаблони проектування відповідають за ефективну взаємодію між об'єктами, розподіл обов'язків та організацію алгоритмів у системі. Вони допомагають налаштувати гнучкі зв'язки між різними компонентами програми, зменшуючи їхню залежність один від одного. У цій лабораторній роботі було розглянуто три ключові патерни цієї групи, а саме Ітератор, Посередник та Спостерігач.

Ітератор надає уніфікований спосіб послідовного доступу до всіх елементів складеного об'єкта, приховуючи його внутрішнє улаштування та особливості збереження даних від клієнтського коду. Посередник, або Медіатор, виступає центральним вузлом, який інкапсулює логіку взаємодії цілої групи об'єктів. Це звільняє інші класи від необхідності явно посилатися один на одного напряму, що значно спрощує архітектуру та забезпечує слабку зв'язаність системи. Спостерігач визначає залежність типу один до багатьох, за якої зміна стану одного головного об'єкта автоматично викликає оповіщення та оновлення всіх пов'язаних із ним залежних об'єктів-підписників.

1.4.4 Висновки, що відображують результати виконання роботи та їх критичний аналіз

У ході лабораторної роботи на мові Python було успішно виконано програмну реалізацію трьох поведінкових шаблонів проектування. Завдяки патерну Ітератор було організовано послідовний перебір елементів колекції через методи `has_next()` та `next()` без прямого розкриття структури внутрішнього сховища об'єктів. Через Посередник вдалося повністю розірвати прямі зв'язки між об'єктами-колегами, оскільки вони стали обмінюватися подіями виключно через центральний об'єкт `ConcreteMediator` за допомогою методу `notify()`, який координує подальшу реакцію системи. Спостерігач продемонстрував динамічне управління залежностями, де додавання через `attach()`, видалення через `detach()` та автоматичне сповіщення об'єктів про зміни стану суб'єкта відбуваються незалежно від їхніх конкретних реалізацій.

Критичний аналіз побудованої архітектури показує, що використання поведінкових патернів помітно підвищило гнучкість системи, спростило її

масштабування та дозволило дотримуватися принципів проектування SOLID. Проте основним архітектурним компромісом є те, що складний потік керування стає важче відстежувати під час виконання програми, оскільки виклики проходять через додаткові рівні абстракцій. Зокрема, для медіатора існує ризик перетворення центрального класу на занадто великий і перевантажений логікою об'єкт при сильному розширенні системи. Проте для великих проектів такі витрати є цілком виправданими, оскільки вони усувають жорстку зв'язаність коду.

Приклад роботи

```
PS C:\Users\krivo\Desktop\construction\lab10> python .\main.py
ЛАБОРАТОРНАЯ РАБОТА №8: Поведенческие шаблоны проектирования
```

```
--- Демонстрация шаблона Iterator ---
```

```
Обход коллекции книг с помощью Итератора:
```

```
Книга 1
```

```
Книга 2
```

```
Книга 3
```

```
--- Демонстрация шаблона Mediator ---
```

```
Клиент запускает операцию B у Colleague1, посредник реагирует:
```

```
Colleague1 does something B.
```

```
Mediator reacts on Colleague1 and triggers Colleague2: B
```

```
Colleague2 does something else.
```

```
Клиент запускает операцию C у Colleague2, посредник реагирует:
```

```
Colleague2 does something C.
```

```
Mediator reacts on Colleague2 and triggers Colleague1: C
```

```
Colleague1 does something A.
```

```
Mediator reacts on Colleague1 and triggers Colleague2: A
```

```
Colleague2 does something else.
```

```
--- Демонстрация шаблона Observer ---
```

```
Субъект изменяет состояние (10):
```

```
Subject: Changing state to 10
```

```
Observer Пользователь A: Subject state updated to 10
```

```
Observer Пользователь B: Subject state updated to 10
```

```
Субъект изменяет состояние (20), Пользователь A отписывается:
```

```
Subject: Changing state to 20
```

```
Observer Пользователь B: Subject state updated to 20
```

Контрольні запитання

1.5.1 Призначення шаблонів поведінки

Вони керують алгоритмами, розподілом обов'язків та налагодженням взаємодії між об'єктами в системі.

1.5.2 Характеристика шаблонів

Ітератор перебирає елементи колекції, Посередник організує спілкування через один центр, Спостерігач повідомляє підписників про зміни, Стратегія міняє алгоритми на льоту, Ланцюжок відповідальності передає запит по черзі, а Відвідувач додає нові операції до класів без їх зміни.

1.5.3 - 1.5.5 Шаблон Iterator

Він послідовно перебирає елементи колекції, ховаючи її внутрішню структуру від клієнта. Складається з Aggregate (інтерфейс колекції), ConcreteAggregate, Iterator (інтерфейс обходу) та ConcreteIterator. При реалізації клієнт викликає методи на кшталт next() та has_next(). Результат дозволяє однаково обходити абсолютно різні типи сховищ даних.

1.5.6 - 1.5.8 Шаблон Mediator

Інкапсулює взаємодію групи об'єктів у єдиному центрі, щоб класи не посилалися один на одного напряму. Учасники це Mediator, ConcreteMediator та класи-колеги Colleague. Особливість у тому, що колеги шлють повідомлення посереднику через notify(), а він уже сам розподіляє їх. Результат значно зменшує кількість прямих зв'язків у системі.

1.5.9 - 1.5.11 Шаблон Observer

Дозволяє автоматично сповіщати безліч об'єктів-підписників про зміну стану одного головного суб'єкта. Включає Subject (суб'єкт), ConcreteSubject, Observer (інтерфейс спостерігача) та ConcreteObserver. Суб'єкт динамічно додає підписників через масив і викликає їхні методи оновлення при змінах. Результат забезпечує гнучкий та слабкий зв'язок між компонентами програми.

1.5.12 Спільне використання

Ітератор часто обходить список підписників у Спостерігачі, а сам Посередник може використовувати Спостерігача, щоб динамічно ловити події від колег.

1.5.13 - 1.5.15 Шаблон Strategy

Визначає родину алгоритмів, інкапсулює кожен з них і робить їх взаємозамінними під час виконання програми. Учасники це Context (контекст), Strategy (інтерфейс алгоритму) та ConcreteStrategy (варіанти логіки). Клієнт просто передає потрібну стратегію в контекст, який делегує їй виконання роботи. Результат дозволяє легко міняти поведінку об'єкта без використання великих конструкцій if-else.

1.5.16 - 1.5.18 Шаблон Chain of Responsibility

Дозволяє передавати запити послідовно по ланцюжку обробників, поки один із них не виконає дію. Складається з Handler (інтерфейс обробника) та ConcreteHandler (конкретні ланки). Кожен обробник зберігає посилання на наступного і або виконує запит сам, або передає його далі по ланцюгу. Результат сильно послаблює зв'язаність між відправником запиту та отримувачем.

1.5.19 - 1.5.21 Шаблон Visitor

Дозволяє додавати нові операції до групи класів, не змінюючи самі ці класи. Учасники це Visitor (відвідувач), ConcreteVisitor, Element (інтерфейс елемента) та ConcreteElement. Елементи приймають відвідувача через метод асерт, а відвідувач виконує специфічну для цього класу операцію. Результат спрощує додавання нових операцій, але ускладнює розширення самих типів елементів при зміні ієрархії.

1.5.22 Спільне використання

Стратегію можна використовувати для динамічного налаштування ланок у Ланцюжку відповідальності, а Відвідувач може обходити складні деревоподібні структури об'єктів за допомогою Ітератора.